

Eliminación Gaussiana y Sustitución Inversa

Traduciendo álgebra lineal matricial
a eficiencia computacional

BY: Andrés Arrieta

$$\begin{bmatrix} 1 & \cdot & \cdot & \cdot \\ \emptyset & 1 & \cdot & \cdot \\ \emptyset & \emptyset & 1 & \cdot \\ \emptyset & \emptyset & \emptyset & 1 \end{bmatrix}$$

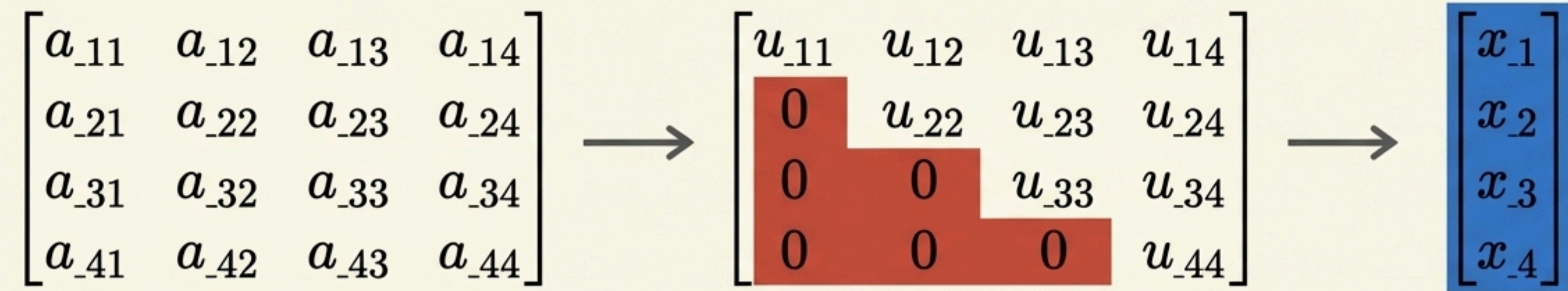
$$a_{33}x_3 = b_3$$

$$a_{22}x_2 + a_{23}x_3 = b_2$$

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1$$

Una estrategia secuencial y directa

Un método de dos fases que evita la inversión de matrices, siendo más rápido y preciso para la máquina.



1. Eliminación Gaussiana

Simplificar el sistema reduciendo la matriz a una forma triangular superior.

2. Sustitución Inversa (Hacia Atrás)

Resolver las variables secuencialmente desde la base del sistema hasta la cima.

Eliminación Gaussiana

Partiendo de que en física aparecen problemas de sistema de ecuaciones lineales de la forma:

$$\begin{aligned} A\mathbf{x} &= \mathbf{b} \\ A &= \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{pmatrix} \\ \mathbf{x} &= \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \\ \mathbf{b} &= \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} \end{aligned}$$

Mediante operaciones entre líneas hacemos que la matriz original opte por una forma de matriz triangular superior.

$$F_j = F_j - a_{ij}F_i$$

F_i : Fila pivote F_j : Fila a modificar a_{ij} : Multiplicador

Transformando así el sistema original en uno equivalente con matriz triangular superior de la forma:

$$\begin{bmatrix} \mathbf{1} & a_{01} & a_{02} & a_{03} \\ \mathbf{0} & \mathbf{1} & a_{12} & a_{13} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} & a_{23} \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{1} \end{bmatrix}$$

La diagonal principal se normaliza a unos y los elementos inferiores se reducen a cero. El sistema se simplifica sin alterar su equivalencia.

La base de la sustitución inversa

En este punto es posible resolver el sistema mediante sustitución inversa resolviendo el sistema empezando desde la ultima ecuación hacia la primera

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

Dada la estructura la ultima ecuación se vuelve trivial, revelando así la primera respuesta directa.

$$a_{33}x_3 = b_3$$

$$a_{22}x_2 + a_{23}x_3 = b_2$$

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1$$

- Permitiendo así aislar la última variable de esta manera cada solución encontrada se usa para calcular las anteriores.

$$x_3 = \frac{b_3}{a_{33}}$$

$$x_2 = \frac{b_2 - a_{23}x_3}{a_{22}}$$

$$x_1 = \frac{b_1 - a_{12}x_2 - a_{13}x_3}{a_{11}}$$

Forma general :

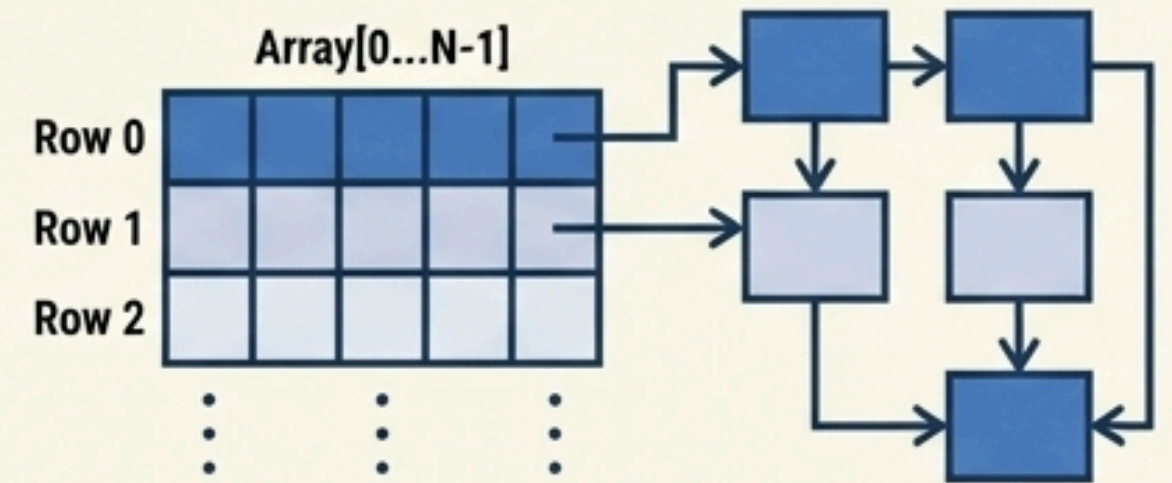
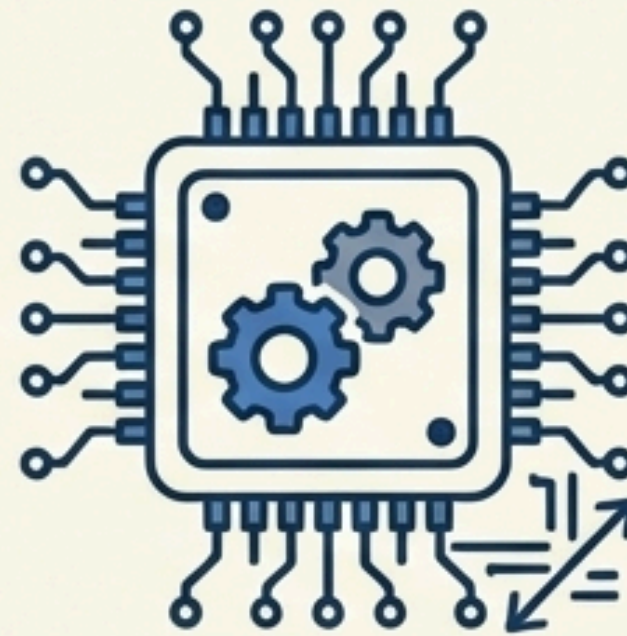
$$x_i = \frac{b_i - \sum_{j=i+1}^n a_{ij}x_j}{a_{ii}}$$

$i = n, n - 1, \dots, 1$ Esto significa que las variables se calculan de atrás hacia adelante.

De la pizarra a la máquina

Hand-drawn mathematical formulas on a chalkboard background:

- $\int f(x) dx = \sum_{k=1}^n f(x) = 1$
- $f \cdot l = \sum_{k=0}^{\infty} x_i$ (with a red arrow pointing to "scriblers")
- A matrix: $\begin{bmatrix} 1 & 2 & 1 & 2 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 2 & 1 \end{bmatrix}$ (with a red arrow pointing to "linear algebra")
- A system of equations: $\begin{cases} a_1 = tz \mathbb{R}^n \\ (-b_2)^n + a_2 t \end{cases}$



- **Índices base cero:** Arrays alineados con el estándar de Python (0 a N-1).
- **Estructuras de iteración:** Ciclos `for` que replican secuencialmente la eliminación fila por fila.
- **Operaciones vectorizadas:** Aceleración del cálculo procesando aritmética en filas enteras simultáneamente.

Implementación: Eliminación Gaussiana

```
# Matriz
A = np.array([
    [2.0, 1.0, -1.0],
    [-3.0, -1.0, 2.0],
    [-2.0, 1.0, 2.0]
])
# Vector de terminos independientes
b = np.array([8.0, -11.0, -3.0])
```

Matriz de coeficientes del sistema de ecuaciones lineales.

Este ciclo recorre las filas de la matriz en donde i representa la fila pivote para eliminar los elementos debajo del pivote

```
for i in range(N):
    for j in range(i+1, N):
        multiplicador = A[j][i] / A[i][i]

        for k in range(i, N):
            A[j][k] = A[j][k] - multiplicador * A[i][k]

        b[j] = b[j] - multiplicador*b[i]
```

Recorre las filas que estan debajo de la fila pivote donde la variable j representa la fila a modificar para eliminar el elemento debajo del pivote.

El factor de eliminacion(multiplicador) indica cuánto de la fila pivote debe restarse de la fila actual para eliminar el elemento debajo del pivote.

Matemáticamente corresponde al multiplicador: $m_{ji} = \frac{a_{ji}}{a_{ii}}$

Este ciclo recorre las columnas de la matriz la variable k representa la columna que se está actualizando.

Implementación: Sustitución Inversa

```
for i in range(N-1, -1, -1):  
  
    suma = 0  
  
    for j in range(i+1, N):  
        suma += A[i][j]*x[j]  
  
    x[i] = (b[i] - suma) / A[i][i]
```

Este ciclo recorre las ecuaciones desde la última hacia la primera.

Este ciclo calcula la contribución de las variables que ya fueron calculadas.

Se despeja la variable correspondiente usando la ecuación:

$$x_i = \frac{b_i - \sum_{j=i+1}^n a_{ij}x_j}{a_{ii}}$$

Caso de prueba: Un sistema multidimensional

Verificación del modelo computacional de un sistema de ecuaciones de 3 variables (x,y,z).

$$2x + y - z = 8$$

$$-3x - y + 2z = -11$$

$$-2x + y + 2z = -3$$

```
matriz original
```

```
[[ 2.  1. -1.]  
 [-3. -1.  2.]  
 [-2.  1.  2.]]
```

```
vector terminos independientes
```

```
[ 8. -11. -3.]
```

Ejecución y resultado

```
Matriz triangular superior obtenida:
```

```
[[ 2.   1.  -1. ]  
 [ 0.   0.5  0.5]  
 [ 0.   0.  -1. ]]
```

```
Vector solución del sistema obtenido:
```

```
[ 2.  3. -1.]
```

Siendo entonces la solución del sistema:

$$x=2 \quad ; \quad y=3 \quad ; \quad z=-1$$

Evaluación del algoritmo

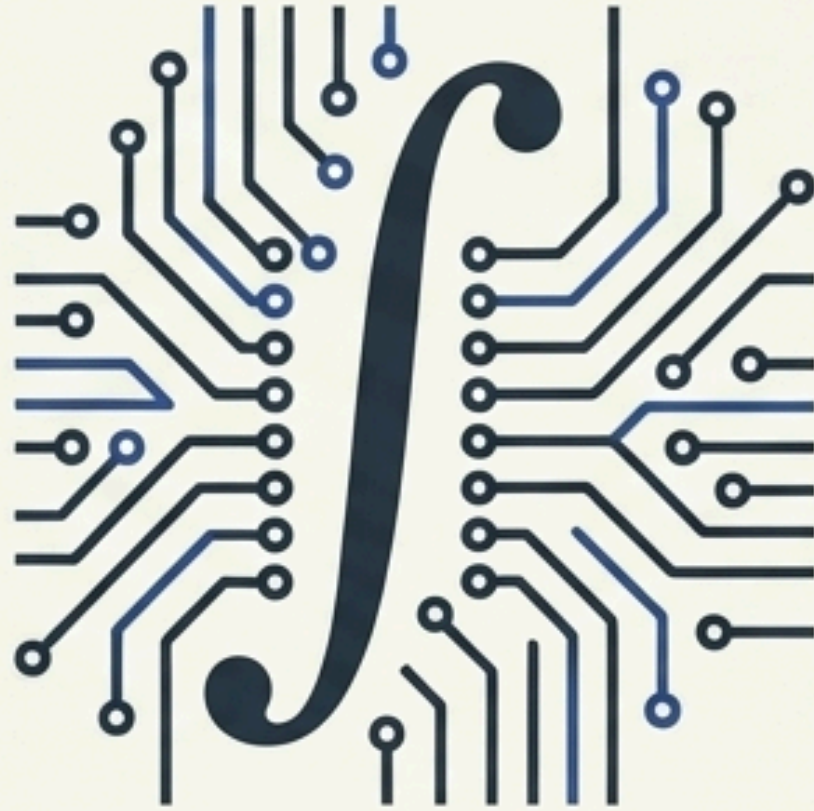
Ventajas Computacionales

- **Directo y secuencial:** Elimina la pesada e ineficiente carga numérica de calcular matrices inversas (A^{-1}).
- **Optimización vectorial:** Aprovecha la capacidad nativa de Python para procesar aritmética en filas enteras simultáneamente.
- **Alta precisión:** Al minimizar la cantidad de operaciones complejas, reduce la propagación de errores numéricos.

Consideraciones Críticas

- **Riesgo de División por Cero:** El algoritmo fallará si un elemento diagonal principal es inherentemente cero durante la normalización.
- **Solución Requerida:** Sistemas con pivotes nulos exigen un paso adicional de 'pivoteo parcial' (intercambio de filas) antes de dividir.

El estándar fundamental



La Eliminación Gaussiana acoplada a la Sustitución Inversa ilustra el núcleo de la **física computacional**: tomar un principio algebraico sólido, estructurarlo sistemáticamente y traducirlo a la lógica nativa de la máquina.

Una herramienta robusta e indispensable para modelar el mundo real.